#Day 7: Application

A software developed to solve a specific problem or perform a task for users.

Frontend

- User interface (UI) of the application.
- Takes user input and displays output.
- Technologies: HTML, CSS, JavaScript, React, Angular.

Backend

- Handles business logic.
- Processes requests from the frontend.
- Connects to the database and returns responses.
- Example: Spring Boot.

Database

- Stores application data permanently.
- Examples: MongoDB, MySQL, PostgreSQL, Supabase.
- Defines the schema (structure of stored data).

Cloud Database

- Database hosted on the internet.
- Accessible from anywhere.
- Example: Supabase.

Schema

- Blueprint of the database.
- Defines tables/collections, fields, and data types.

User Management APIs

The backend should provide the following APIs:

- @Register
- @Login
- @GetUser
- @Modify/UpdateUser
- @DeleteUser

These APIs perform CRUD (Create, Read, Update, Delete) operations on user data.

Database Setup

The application requires a database to store user details.

Possible databases:

- MongoDB
- MySQL
- PostgreSQL
- Supabase (Cloud Database)

Database Responsibilities

- Store user details
- Define the user schema
- Save encrypted passwords
- Retrieve user information during login

User Registration Flow

Home Page

↓

Register

↓

Enter Email

↓

Enter Password

↓

Re-enter Password

↓

Validate Email

↓

Check Passwords Match

↓

Encrypt Password

↓

Enable Submit Button

↓

Call Register API

1. Register API Contract

Endpoint

POST /register

Request

```
{
  "email": "user@gmail.com",
  "password": "encrypted_password"
}
```

Fields:

- Email
- Password (Encrypted)

2. Register Backend Flow

Receive Request

↓

Validate Email

↓

Validate Password

↓

Encrypt Password

↓

Connect Database

↓

Store User Details

↓

Return Response

3. Register Response

Success


```
{
  "status":"OK",
  "statusCode":200,
  "message":"User Registered Successfully"
}
```


Error Status Codes

200 → Success

400 → Bad Request

404 → Resource Not Found

500 → Internal Server Error

4. After Registration

If registration succeeds

Route User → Login Page

Otherwise

Display Error Message

5. Login Flow

Login

↓

Enter Email

↓
Enter Password
↓
Validate Email
↓
Encrypt Password
↓
Call Login API

6. Login API Contract
Endpoint
POST /login
Request

```
{
  "email":"user@gmail.com",
  "password":"encrypted_password"
}
```

7. Login Backend Flow
Receive Login Request
↓
Validate Email
↓
Encrypt Password
↓
Connect Database
↓
Compare Password
↓
Authentication Success
↓
Generate JWT Token
↓
Return Response
8. Login Response

```
{
  "status":"OK",
  "statusCode":200,
  "message":"Authentication Successful"
```

}
or
{
  "status":"ERROR",
  "statusCode":400,
  "message":"Authentication Failed"
}
9. Dashboard Navigation
If authentication succeeds
Login
  ↓
Generate JWT
  ↓
Return Token
  ↓
Navigate to Dashboard
10. JWT Authentication
JWT = JSON Web Token
Purpose:
- Authenticate users
- Identify logged-in users
- Protect secured APIs

Flow
Login
  ↓
JWT Generated
  ↓
Frontend Stores Token
  ↓
Token Sent with Every Request
  ↓
Backend Verifies JWT
  ↓
Access Granted
11. Backend Register Process
Frontend
   ↓
POST /register

↓
Controller
↓
Validate Input
↓
Encrypt Password
↓
Service Layer
↓
Repository
↓
Database
↓
Response

12. Backend Login Process
Frontend
↓
POST /login
↓
Controller
↓
Validate Email
↓
Encrypt Password
↓
Compare with Database
↓
Generate JWT
↓
Return Response
13. User Schema
User

id
email
password
createdAt

updatedAt

Password should always be stored in encrypted form.

13. API Request & Response

Request

Client sends

Email

Password

Response

Server returns

Status

Status Code

Message

JWT Token (after login)

14. HTTP Status Codes

Status Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error